Applied Economics

Data and Plotting

Pello Aspuru

Winter 2026/2027

Introduction

Previous sessions:

- Set up R
- Download, create and merge datasets.

Today: Visualize data with graphs.

- Review types of variables, how they match to graph types.
- Learn how to plot data with ggplot2.
- Discuss what makes a good graph and how to improve them.

Data Plots and Econometric Analysis

Advantages	Disadvantages
Easy to read	Misleading: axis scales, color use...
Shows variables' distribution: outliers, skewness...	Omitted variables: Only shows 2–3 at once.
Fewer assumptions: no functional form choice.	No causal effect, no significance test.

Plots and econometric analysis are **complements**:

- Plots help to understand variables' features and correlates → Choose the best econometric model.
- Econometric analysis to establish causal relationships and test their significance.

The author holds significant **discretion**: be honest with yourself and your audience.

- Decide the purpose of the graph.
- Tradeoffs: what to show, how to show it, and what to skip.

Data Plots and Econometric Analysis

Advantages	Disadvantages
Easy to read	Misleading: axis scales, color use...
Shows variables' distribution: outliers, skewness...	Omitted variables: Only shows 2–3 at once.
Fewer assumptions: no functional form choice.	No causal effect, no significance test.

Plots and econometric analysis are **complements**:

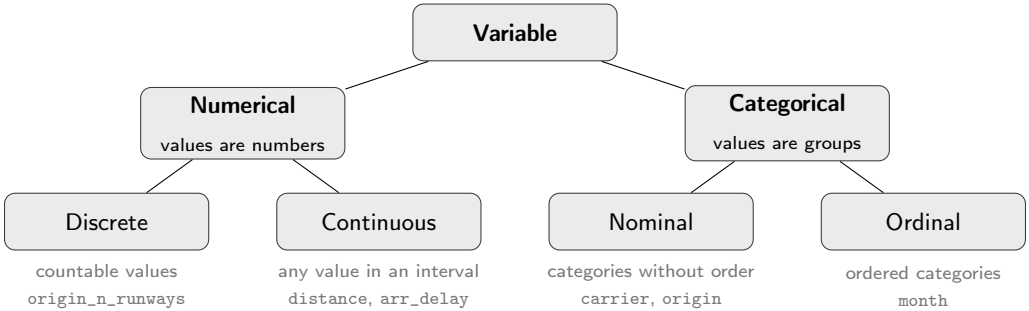
- Plots help to understand variables' features and correlates $\rightarrow$ Choose the best econometric model.
- Econometric analysis to establish causal relationships and test their significance.

The author holds significant **discretion**: be honest with yourself and your audience.

- Decide the purpose of the graph.
- Tradeoffs: what to show, how to show it, and what to skip.

Types of Variables in a Dataset

```
df_flights <- read.csv("data/df_flights_enriched.csv")
```



Types of Variables and Their Graphs

Graph	Variables	Shows	ggplot2
Histogram	1 continuous	distribution of a variable	`geom_histogram()`
Bar chart	1 categorical	number of observations per category	`geom_bar()`
Box plot	1 continuous + 1 categorical	distribution within each category	`geom_boxplot()`
Scatter plot	2 continuous	relationship between two variables	`geom_point()`
Line graph	1 continuous + time	evolution of a variable over time	`geom_line()`

Plotting Data with ggplot2

ggplot2 is the most widely used R package for graphs. A plot is built in **layers** joined with +:

- `ggplot(data, aes(...))`: the dataset and which variables go on each axis (or colour, fill...).
- `geom_...()`: the type of graph (histogram, points, lines...).
- `labs()`, `theme()`, ...: labels and appearance.

Minimal example:

```
# Histogram of arrival delays
ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(bins = 30) +
  labs(x = "Arrival delay (minutes)", y = "Frequency")
```

Note

Cancelled flights have no `arr_delay` or `air_time`, so ggplot warns that it removed rows with missing values. The plot is still drawn.

Plotting Data with ggplot2

ggplot2 is the most widely used R package for graphs. A plot is built in **layers** joined with +:

- `ggplot(data, aes(...))`: the dataset and which variables go on each axis (or colour, fill...).
- `geom_...()`: the type of graph (histogram, points, lines...).
- `labs()`, `theme()`, ...: labels and appearance.

Minimal example:

```
# Histogram of arrival delays
ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(bins = 30) +
  labs(x = "Arrival delay (minutes)", y = "Frequency")
```

Note

Cancelled flights have no `arr_delay` or `air_time`, so ggplot warns that it removed rows with missing values. The plot is still drawn.

Plotting Data with ggplot2

ggplot2 is the most widely used R package for graphs. A plot is built in **layers** joined with +:

- `ggplot(data, aes(...))`: the dataset and which variables go on each axis (or colour, fill...).
- `geom_...()`: the type of graph (histogram, points, lines...).
- `labs()`, `theme()`, ...: labels and appearance.

Minimal example:

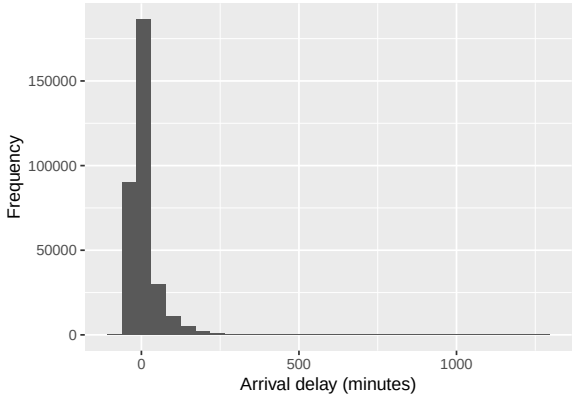
```
# Histogram of arrival delays
ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(bins = 30) +
  labs(x = "Arrival delay (minutes)", y = "Frequency")
```

Note

Cancelled flights have no `arr_delay` or `air_time`, so ggplot warns that it removed rows with missing values. The plot is still drawn.

Histogram of arrival delays

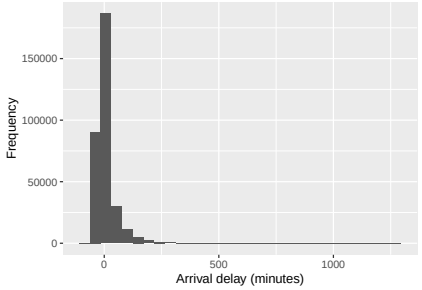
```
# Histogram of arrival delays
ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(bins = 30) +
  labs(x = "Arrival delay (minutes)",
       y = "Frequency")
```



Histogram

Represents the distribution of a **single continuous variable**.

- **Horizontal axis**: the values of the variable, divided into intervals of equal width (**bins**).
- **Vertical axis**: the **number of observations** (frequency) whose value falls in each bin.
- The **height** of each bar is the count.



Decision: The number of bins (`bins = 30`) changes how detailed the shape looks.

Vibecoding Graphs

Before LLMs → graph scarcity. After LLMs → graph congestion.

Think before vibecoding: what is the purpose of the graph? What is the best way to show it?

I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

- The programming language, the package.
- The dataset and variable names.
- The type of graph.

Vibecoding Graphs

Before LLMs → graph scarcity. After LLMs → graph congestion.

Think before vibecoding: what is the purpose of the graph? What is the best way to show it?

I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

- The programming language, the package.
- The dataset and variable names.
- The type of graph.

Vibecoding Graphs

Before LLMs → graph scarcity. After LLMs → graph congestion.

Think before vibecoding: what is the purpose of the graph? What is the best way to show it?

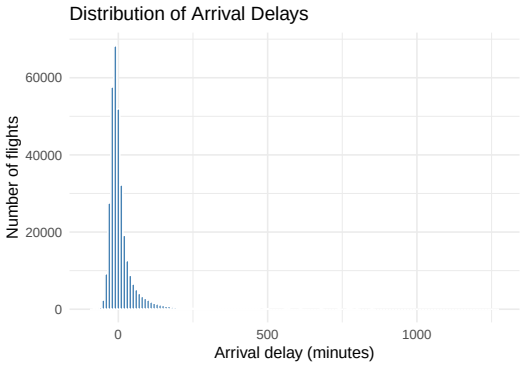
I am working on R using the `ggplot2` graphs package. I have a continuous variable `df_flights$arr_delay`, representing the arrival delay of flights that departed from NYC airports. The dataset is based on the "flights" dataset from the `nycflights13` R package. Write an R script using ggplot2 that shows a histogram of this variable.

- The programming language, the package.
- The dataset and variable names.
- The type of graph.

First Attempt

```
library(ggplot2)

ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 10,
                 fill = "steelblue",
                 color = "white",
                 na.rm = TRUE) +
  labs(title = "Distribution of Arrival Delays",
       x = "Arrival delay (minutes)",
       y = "Number of flights") +
  theme_minimal()
```

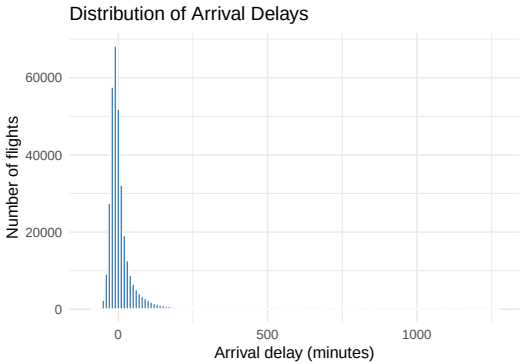


How would you improve this histogram?

First Attempt

```
library(ggplot2)

ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 10,
                 fill = "steelblue",
                 color = "white",
                 na.rm = TRUE) +
  labs(title = "Distribution of Arrival Delays",
       x = "Arrival delay (minutes)",
       y = "Number of flights") +
  theme_minimal()
```



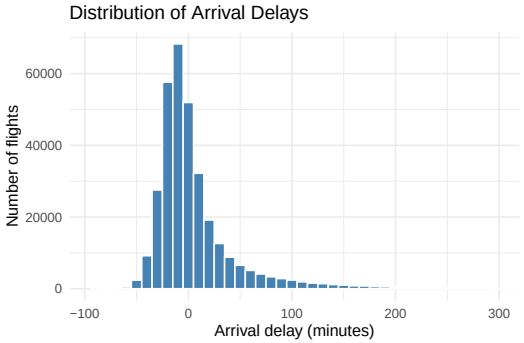
How would you improve this histogram?

Refining the Histogram

I want to improve the histogram of arrival delays. I want to reduce the horizontal axis range to a maximum of 300 minutes. Add a footnote that explains this axis range limitation.

```
n_above <- sum(df_flights$arr_delay > 300,
               na.rm = TRUE)

ggplot(df_flights, aes(x = arr_delay)) +
  geom_histogram(binwidth = 10,
                 fill = "steelblue",
                 color = "white",
                 na.rm = TRUE) +
  coord_cartesian(xlim = c(NA, 300)) +
  labs(title = "Distribution of Arrival Delays",
       x = "Arrival delay (minutes)",
       y = "Number of flights",
       caption = paste0(
         "Note: horizontal axis limited to 300 ",
         "minutes. ", n_above, " flights with ",
         "longer delays are not shown.")) +
  theme_minimal()
```

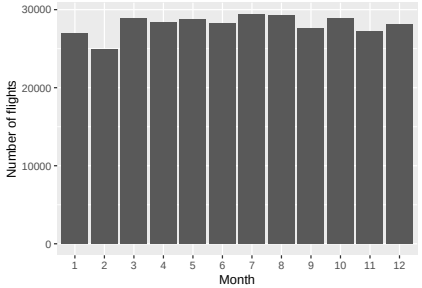


Note: horizontal axis limited to 300 minutes. 611 flights with longer delays are not shown.

Bar Chart

Represents the frequency of a **single categorical variable**.

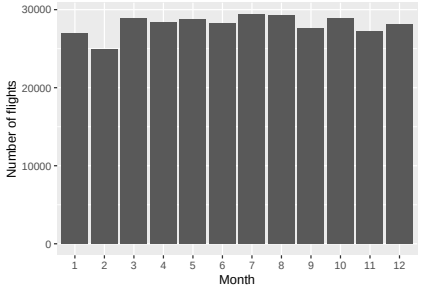
- **Horizontal axis**: the categories of the variable (months of the year), one bar per category.
- **Vertical axis**: the **number of observations** in each category.
- The **height** of each bar is the observation count.



Bar Chart

Represents the frequency of a **single categorical variable**.

```
# Bar chart of flights per month
ggplot(df_flights,
       aes(x = factor(month))) +
  geom_bar() +
  labs(x = "Month",
       y = "Number of flights")
```

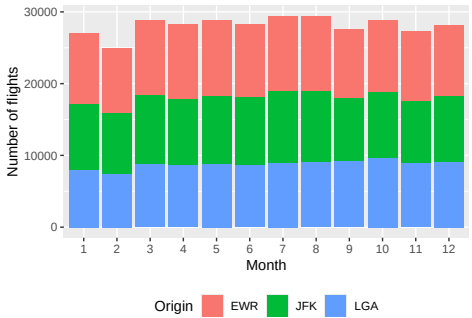


Bar Chart: Adding Categories with Colours

A **second categorical variable** can be shown with the **colour** of the bars.

- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- Decision: bars can be **side by side** or **stacked**.

```
# Stacked (default)
ggplot(df_flights,
       aes(x = factor(month),
           fill = origin)) +
  geom_bar() +
  labs(x = "Month",
       y = "Number of flights",
       fill = "Origin")
```

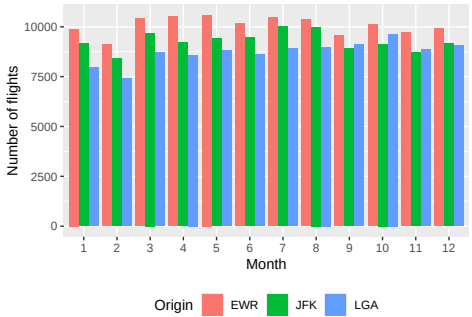


Bar Chart: Adding Categories with Colours

A **second categorical variable** can be shown with the **colour** of the bars.

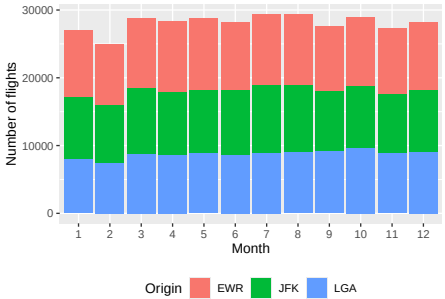
- One colour per airport, shown in the **legend**.
- Each colour counts the flights in a month **and** from one airport.
- Decision: bars can be **side by side** or **stacked**.

```
# Side by side
ggplot(df_flights,
       aes(x = factor(month),
           fill = origin)) +
  geom_bar(position = "dodge") +
  labs(x = "Month",
       y = "Number of flights",
       fill = "Origin")
```

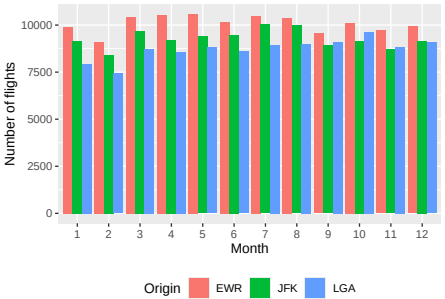


Bar Chart Type Discussion

Stacked (default)



Side by side (`position = "dodge"`)



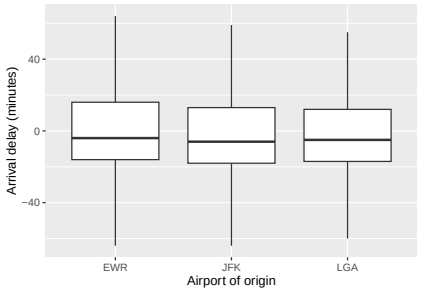
Which do you prefer? Why?

Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

Decision: should we include outliers?

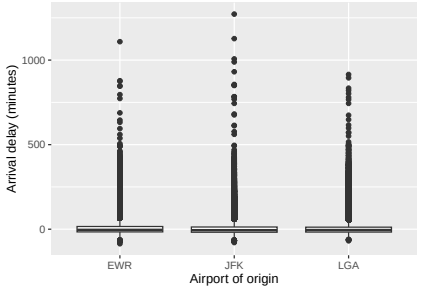


Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

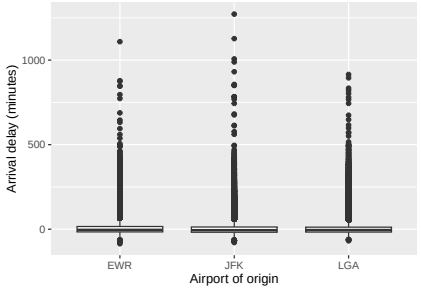
Decision: should we include outliers?



Box Plot

Summary statistics of a **continuous variable** (delay time) within each **category** (airport of origin).

- **Horizontal axis**: the categories of the variable.
- **Vertical axis**: continuous variable.
- The **line** inside the box is the median.
- The **box** ranges from the 25th to the 75th percentile.
- The **whiskers** extend to the most extreme values that lie within 1.5 box heights.
- **Points** outside the whiskers.

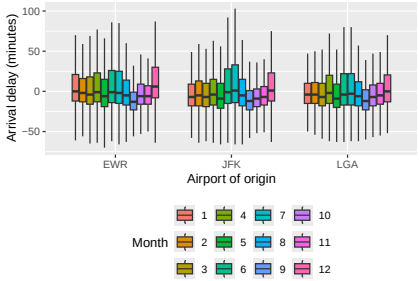


Decision: should we include outliers?

Box Plot: Adding Categories with Colours

A **second categorical variable** (month of the year) can be shown with the **colour** of the boxes.

- One colour per month of the year, shown in the **legend**.
- Within each airport, the boxes of each month are placed **side by side**.
- Each box summarises the delays of the flights from one airport **and** in one month.
- With 12 categories, that is 36 boxes: hard to read and to compare.

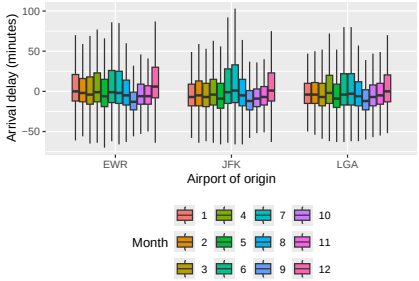


What does the graph show? How can we improve it?

Box Plot: Adding Categories with Colours

A **second categorical variable** (month of the year) can be shown with the **colour** of the boxes.

- One colour per month of the year, shown in the **legend**.
- Within each airport, the boxes of each month are placed **side by side**.
- Each box summarises the delays of the flights from one airport **and** in one month.
- With 12 categories, that is 36 boxes: hard to read and to compare.

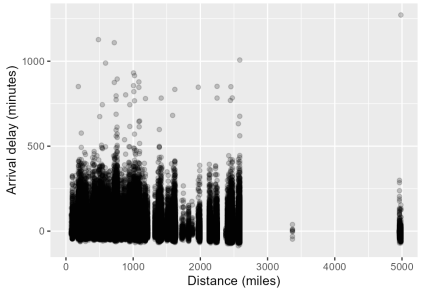


What does the graph show? How can we improve it?

Scatter Plot

Represents the correlation between **two continuous variables** (distance and arrival delay).

- **Horizontal axis**: the first continuous variable (distance in miles).
- **Vertical axis**: the second continuous variable (arrival delay in minutes).
- Each **point** is one observation (one flight), placed at its distance and its delay.
- With many observations, points overlap: transparency (`alpha = 0.2`) shows where they concentrate.

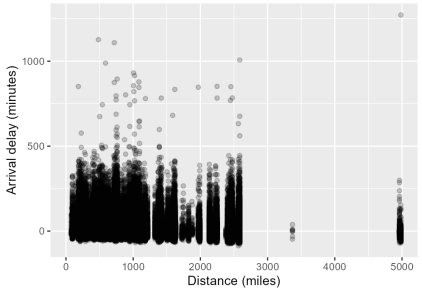


Scatter Plot

Represents the correlation between **two continuous variables** (distance and arrival delay).

```
# Scatter plot of delay against distance
ggplot(df_flights,
       aes(x = distance,
           y = arr_delay)) +
  geom_point(alpha = 0.2) +
  labs(x = "Distance (miles)",
       y = "Arrival delay (minutes)")
```

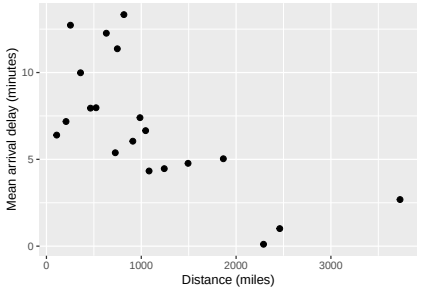
What does the plot show?



Binscatter Plot

When scatterplots are too crowded, they can be summarized with a binscatter plot.

- **Horizontal axis**: the first variable (distance), divided into 20 **bins** with the same number of flights each.
- **Vertical axis**: the **mean** of the second variable (arrival delay) within each bin.
- Each **point** is one bin, not one flight.



The hidden pattern **emerges**: longer flights arrive, on average, with shorter delays.

Vibecoding a Binscatter Plot

I have this scatterplot that is too crowded [paste the previous code here]. I want to turn it into a binscatter plot with 20 bins. Please reflect any other arbitrary decisions in the graph footnote.

```
d <- subset(df_flights, !is.na(arr_delay))
brk <- quantile(d$distance,
                probs = seq(0, 1, 0.05))
d$bin <- cut(d$distance, unique(brk),
             include.lowest = TRUE)
bins <- aggregate(cbind(distance, arr_delay)
                  ~ bin, data = d, FUN = mean)
```

```
ggplot(bins, aes(x = distance,
                 y = arr_delay)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       caption = paste(
         "Note: 20 distance bins with roughly equal",
         "numbers of flights. Points are bin means.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```

Vibecoding a Binscatter Plot

I have this scatterplot that is too crowded [paste the previous code here]. I want to turn it into a binscatter plot with 20 bins. Please reflect any other arbitrary decisions in the graph footnote.

```
d <- subset(df_flights, !is.na(arr_delay))
brk <- quantile(d$distance,
                probs = seq(0, 1, 0.05))
d$bin <- cut(d$distance, unique(brk),
             include.lowest = TRUE)
bins <- aggregate(cbind(distance, arr_delay)
                  ~ bin, data = d, FUN = mean)
```

```
ggplot(bins, aes(x = distance,
                 y = arr_delay)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       caption = paste(
         "Note: 20 distance bins with roughly equal",
         "numbers of flights. Points are bin means.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```

How the bins are made:

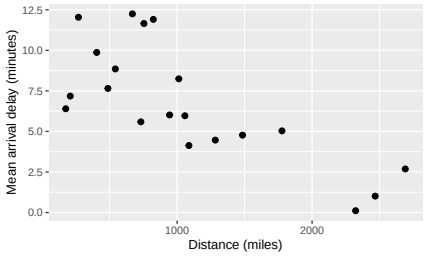
1. Order the flights by distance, drop `NA` values.
2. `brk` breaks at every 5th percentile.
3. `d$bin` assigns each flight to one of the 20 bins.
4. `bins` computes the mean distance and the mean delay.

Vibecoding a Binscatter Plot

I have this scatterplot that is too crowded [paste the previous code here]. I want to turn it into a binscatter plot with 20 bins. Please reflect any other arbitrary decisions in the graph footnote.

```
d <- subset(df_flights, !is.na(arr_delay))
brk <- quantile(d$distance,
                probs = seq(0, 1, 0.05))
d$bin <- cut(d$distance, unique(brk),
             include.lowest = TRUE)
bins <- aggregate(cbind(distance, arr_delay)
                  ~ bin, data = d, FUN = mean)
```

```
ggplot(bins, aes(x = distance,
                 y = arr_delay)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       caption = paste(
         "Note: 20 distance bins with roughly equal",
         "numbers of flights. Points are bin means.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```

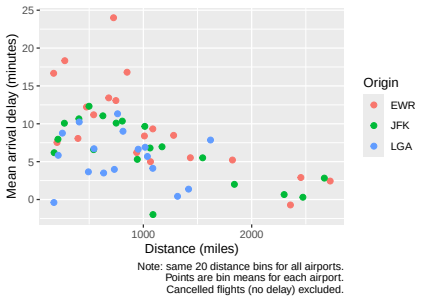


Binscatter improvements: Adding Categories with Colours

Can you improve the previous binscatter by classifying the flights by their origin airport? Each airport should be represented by a different colour. Please add a legend to the graph.

```
# Same bins as before (d, brk, d$bin)
bins <- aggregate(cbind(distance, arr_delay)
                  ~ bin + origin,
                  data = d, FUN = mean)
```

```
ggplot(bins, aes(x = distance,
                 y = arr_delay,
                 colour = origin)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       colour = "Origin",
       caption = paste(
         "Note: same 20 distance bins for all airports.",
         "Points are bin means for each airport.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```

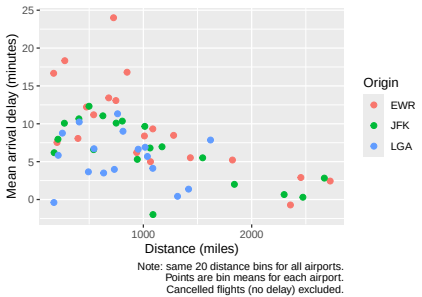


Binscatter improvements: Adding Categories with Colours

Can you improve the previous binscatter by classifying the flights by their origin airport? Each airport should be represented by a different colour. Please add a legend to the graph.

```
# Same bins as before (d, brk, d$bin)
bins <- aggregate(cbind(distance, arr_delay)
                  ~ bin + origin,
                  data = d, FUN = mean)
```

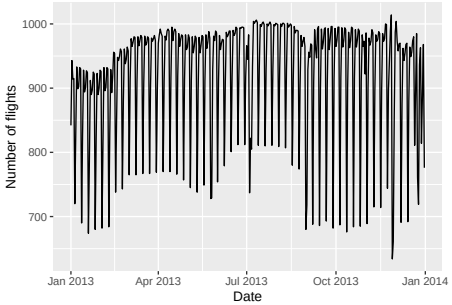
```
ggplot(bins, aes(x = distance,
                 y = arr_delay,
                 colour = origin)) +
  geom_point(size = 2) +
  labs(x = "Distance (miles)",
       y = "Mean arrival delay (minutes)",
       colour = "Origin",
       caption = paste(
         "Note: same 20 distance bins for all airports.",
         "Points are bin means for each airport.",
         "Cancelled flights (no delay) excluded.",
         sep = "\n"))
```



Line Graph

A line graph shows how **one continuous variable evolves over time**.

- **Horizontal axis**: time (here, each day of 2013).
- **Vertical axis**: the value of the variable at each date (here, the number of flights that day).
- Points are **connected in time order**.



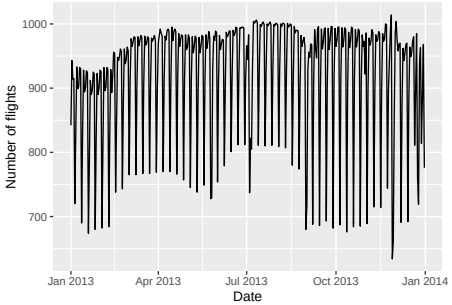
What does the graph show? How can we improve it?

Line Graph

A line graph shows how **one continuous variable evolves over time**.

```
# Count the flights of each day
library(dplyr)
daily <- df_flights |>
  mutate(date = as.Date(paste(year, month,
                              day, sep = "-"))) |>
  group_by(date) |>
  summarise(n = n())
```

```
ggplot(daily, aes(x = date, y = n)) +
  geom_line() +
  labs(x = "Date", y = "Number of flights")
```



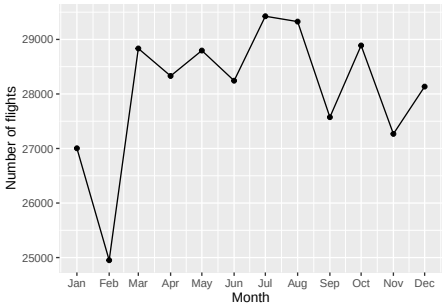
What does the graph show? How can we improve it?

Line Graph: Aggregating to Months

The **time unit** we choose changes what the graph shows.

```
# Count the flights of each month
df_flights <- df_flights |>
  mutate(ym = as.Date(paste(year, month,
                            1, sep = "-")))
monthly <- df_flights |>
  group_by(ym) |>
  summarise(n = n())
```

```
ggplot(monthly, aes(x = ym, y = n)) +
  geom_line() +
  geom_point() +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(x = "Month", y = "Number of flights")
```

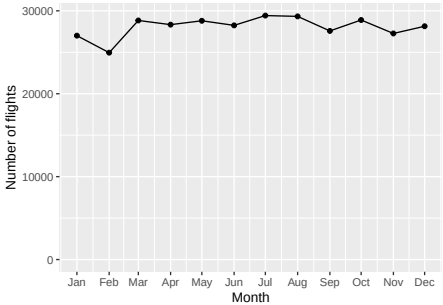


What is happening in February?

Line Graph: Starting the Vertical Axis at Zero

Where the vertical axis starts changes how **large** the changes look.

```
# Same monthly graph, vertical axis from zero
ggplot(monthly, aes(x = ym, y = n)) +
  geom_line() +
  geom_point() +
  expand_limits(y = 0) +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(x = "Month", y = "Number of flights")
```

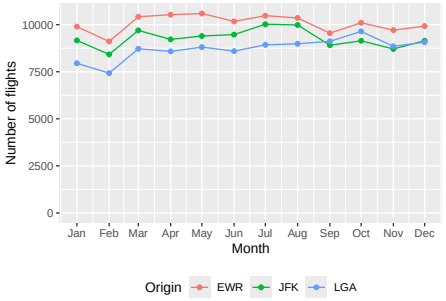


Line Graph: Adding Categories with Colours

A **categorical variable** can be shown with one line per category.

```
# Count the flights of each month and airport
monthly_origin <- df_flights |>
  group_by(ym, origin) |>
  summarise(n = n())
```

```
ggplot(monthly_origin, aes(x = ym, y = n,
                           colour = origin)) +
  geom_line() +
  geom_point() +
  expand_limits(y = 0) +
  scale_x_date(date_breaks = "1 month",
               date_labels = "%b") +
  labs(x = "Month", y = "Number of flights",
       colour = "Origin") +
  theme(legend.position = "bottom")
```

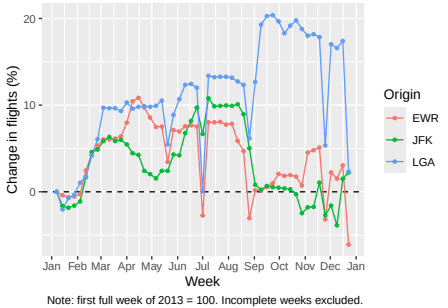


Vibecoding a Line Graph

I want to represent how the number of flights in each airport has changed over the months. For that, I want to aggregate the data to weekly level, then determine the first period as level 100 for each airport, and finally plot the percentage change of flights with respect to the baseline.

```
weekly <- df_flights |>
  # week = date of the Monday that starts each week
  mutate(date = as.Date(paste(year, month, day, sep = "-")),
         week = as.Date(cut(date, "week"))) |>
  group_by(origin, week) |>
  summarise(n = n(), days = n_distinct(date), .groups = "drop") |>
  # drop incomplete weeks (start and end of the year)
  filter(days == 7) |>
  group_by(origin) |>
  mutate(index = 100 * n / first(n),
         pct_change = index - 100) |>
  ungroup()
```

```
ggplot(weekly, aes(x = week, y = pct_change, colour = origin)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line() + geom_point(size = 1) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") +
  labs(x = "Week", y = "Change in flights (%)", colour = "Origin",
       caption = paste("Note: first full week of 2013 = 100.",
                       "Incomplete weeks excluded."))
```

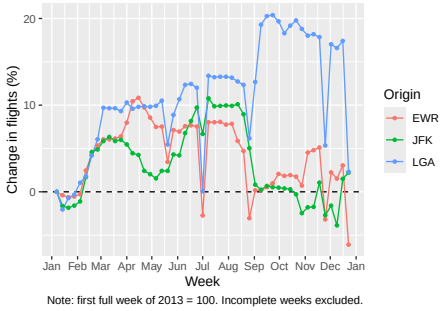


Vibecoding a Line Graph

I want to represent how the number of flights in each airport has changed over the months. For that, I want to aggregate the data to weekly level, then determine the first period as level 100 for each airport, and finally plot the percentage change of flights with respect to the baseline.

```
weekly <- df_flights |>
  # week = date of the Monday that starts each week
  mutate(date = as.Date(paste(year, month, day, sep = "-")),
         week = as.Date(cut(date, "week"))) |>
  group_by(origin, week) |>
  summarise(n = n(), days = n_distinct(date), .groups = "drop") |>
  # drop incomplete weeks (start and end of the year)
  filter(days == 7) |>
  group_by(origin) |>
  mutate(index = 100 * n / first(n),
         pct_change = index - 100) |>
  ungroup()
```

```
ggplot(weekly, aes(x = week, y = pct_change, colour = origin)) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  geom_line() + geom_point(size = 1) +
  scale_x_date(date_breaks = "1 month", date_labels = "%b") +
  labs(x = "Week", y = "Change in flights (%)", colour = "Origin",
       caption = paste("Note: first full week of 2013 = 100.",
                       "Incomplete weeks excluded."))
```



Maps

One variable across regions. Each region (country, state, municipality) is a shape, coloured by the value of a variable (a *choropleth* map). Shapes come as an `sf` object, plotted with `geom_sf()`.

```
library(sf)
library(rnaturalearth)
world <- ne_countries(returnclass = "sf")

# Map of the world
ggplot(world) +
  geom_sf()
```

```
# Countries coloured by a variable
ggplot(world, aes(fill = pop_est)) +
  geom_sf() +
  theme(legend.position = "bottom")
```

- To map your own data, merge it into the `sf` object by a region identifier: `merge(world, df, by = "iso_a3")`.
- `fill` can be continuous (income) or categorical (`continent`).

Maps

One variable across regions. Each region (country, state, municipality) is a shape, coloured by the value of a variable (a *choropleth* map). Shapes come as an `sf` object, plotted with `geom_sf()`.

```
library(sf)
library(rnaturalearth)
world <- ne_countries(returnclass = "sf")

# Map of the world
ggplot(world) +
  geom_sf()
```

```
# Countries coloured by a variable
ggplot(world, aes(fill = pop_est)) +
  geom_sf() +
  theme(legend.position = "bottom")
```

- To map your own data, merge it into the `sf` object by a region identifier: `merge(world, df, by = "iso_a3")`.
- `fill` can be continuous (income) or categorical (`continent`).

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).

Graphs: Summary

Graph	Variables plotted	Adding categories	Variables with categories
Histogram	1	panels (`facet_wrap`) or `fill`	2
Bar chart	1	`fill`	2
Box plot	2	`fill`	3
Scatter plot	2	`colour` or `shape`	3
Line graph	2	`colour` or `linetype`	3

- Panels (`facet_wrap`) work with every graph, and can be combined with colours to add yet another variable.
- Always label the axes and state the units: `labs(x = "...", y = "...")`.
- Later in the course: graphs of **estimates** rather than raw data (fitted regression lines, coefficients with confidence intervals).